Università degli Studi di
Milano - Bicocca

Numerical Simulations of the Ising model through Monte Carlo Markov Chains



Maurizio Errico

Supervisor: **Prof. Mattia Bruno**

Course: Fisica

Academic Year: 2025/2026

Department: Dipartimento di Fisica "Giuseppe Occhialini"

Abstract

This work presents the numerical study of the ferromagnetic Ising model in one, two, and three dimensions to observe phase transitions as a function of the lattice size. The system configurations were obtained by sampling a Markov Chain at fixed temperature generated with the Metropolis-Hastings algorithm following an analysis of its thermalisation and autocorrelation time. The critical temperature was estimated in the two and three dimensional cases by analyzing the finite size scaling of the magnetization. Analogously, the absence of a finite temperature phase transition was found in the one dimensional case, in accordance with analytical results.

Contents

A brief history of the Ising model **2**

1 Theoretical background **3**
1.1 General overview 3
1.2 Phase transitions 4
1.3 Solutions 5
1.3.1 One dimensional case 5
1.3.2 Two dimensional case 5
1.3.3 Three dimensional case 5

2 Methods **6**
2.1 Data representation 6
2.2 Monte Carlo Markov Chains 7
2.2.1 Generation 7
2.2.2 Analysis 10

3 Results **18**
3.1 1D Ising model 18
3.2 2D Ising model 19
3.3 3D Ising model 20

A Appendix **21**

A brief history of the Ising model

The Ising model, named after the physicists Ernst Ising, is a mathematical model of ferromagnetism in statistical mechanics. The model was first proposed by Wilhelm Lenz in 1920 and was later solved by Ising (a student of Lenz) in 1925 for the one-dimensional case. The two-dimensional square-lattice Ising model is one of the simplest statistical models to show a phase transition. Though it is a highly simplified model of a magnetic material, the Ising model can still provide qualitative and sometimes quantitative results applicable to real physical systems. The two dimensional square-lattice Ising model is much harder to solve than the one-dimensional case, and was only given an analytic description much later, by Lars Onsager (1944). No complete analytic solution is known for the three-dimensional case, and only numerical methods are available.

The Ising model has been widely studied in physics, mathematics, and computer science, and has applications in various fields.

Chapter 1

Theoretical background

1.1 General overview

The Ising model is a mathematical model of a ferromagnet. [TODO: add context].

The fundamental assumption of the model is that a solid can be represented as a ferromagnet as an array of fixed ions that only interacting with its closest neighbours (and, optionally, an external magnetic field). Such system is described by the classical Hamiltonian

$$H = -\sum_{\langle i,j \rangle} J_{ij}\sigma_i\sigma_j - \mu\sum_i h_i\sigma_i \tag{1.1}$$

where:

- σ_i is the spin value of the ion of the i-th lattice site.
- J_{ij} is a coupling factor between the i-th and j-th lattice sites.
- The sum over $\langle i, j \rangle$ is meant to be over closest neighbours.
- h_i is the magnetic field in the i-th site.
- μ is the magnetic moment of the ions.

This is the special case of the more general quantum Hamiltonian

$$\hat{H} = -\sum_{\langle i,j \rangle} J_{ij}\hat{\sigma_i^z}\hat{\sigma_j^z} - \mu\sum_i \left(h_{i,x}\hat{\sigma_i^x} + h_{i,y}\hat{\sigma_i^y} + h_{i,z}\hat{\sigma_i^z}\right) \tag{1.2}$$

where σ_i are replaced by the Pauli matrices of the i-th site.

In this text only $h_{i,z}$ will ever be assumed $\neq 0$, and since if both $h_{i,y}$ and $h_{i,y}$ are null then every operator commute with each other in the canonical basis, which means equations 1.1 and 1.2 are the same.

Given that the quantities N = number of particles, $V = Na$ = volume of the lattice (where a is the unimportant lattice constant that can be set to 1) are fixed, if the system is in thermal equilibrium with a reservoir at temperature T then the set of microstates of the system is described by the canonical ensemble. In this case, the probability of the system to be in a microstate σ is given by the Boltzmann distribution

$$P_\beta(\sigma) = \frac{e^{-\beta H(\sigma)}}{Z_\beta} \tag{1.3}$$

where Z_β is the canonical partition function

$$Z = \sum_i e^{-\beta E_i} = \sum_{\sigma_i} e^{-\beta \sum_{\langle i,j \rangle} J_{ij}\sigma_i\sigma_j - \beta\mu \sum_i h_i\sigma_i} \tag{1.4}$$

All the same, given an operator $\hat{O}$ its mean is intended to be

$$\left\langle \hat{O} \right\rangle_\beta = \sum_\sigma \hat{O}(\sigma) P_\beta(\sigma) \tag{1.5}$$

which is the canonical average.

The minus in front of the first sum in equations 1.1 and 1.2 are conventional. Varying the sign of the coefficients J_{ij} two differently behaving systems are described:

1. $J_{ij} > 0$: ferromagnetic
2. $J_{ij} < 0$: antiferromagnetic
3. $J_{ij} = 0$: non-interacting

Historically, the Ising model is conventionally intended to be ferromagnetic. For simplicity sake, J_{ij} along the constants k_B, $\hbar$, h_i, μ will be set $= 1\ \forall i, j$, when not null.

1.2 Phase transitions

It's been stated that the system described by the Ising model is represented by the canonical ensemble, which is governed by the Helmoltz free energy. This, like every other thermodynamic potential, can be expressed as a function of the partition function $Z = Z(T, h)$:

$$F = E - TS = -\frac{\ln(Z)}{N\beta} \tag{1.6}$$

Formally, a phase transition is defined as a non-analyticity in the free energy in the thermodynamic limit. To observe such a non-analyticity one can study the partial derivative

$$M = -\frac{\partial F}{\partial H} \tag{1.7}$$

that takes the name of magnetization.

This is a handy observable in the numerical approach beacuse its average is easily obtainable by averaging the spin values of the lattice sites:

$$\langle M \rangle = \frac{1}{N} \sum_i \sigma_i \tag{1.8}$$

and studying its behaviour as a function of the temperature T and the external magnetic field h one can identify the phase transitions.

It should be noted that this definitions formally hold even in the case of no external magnetic field. This is true only in the thermodynamic limit, where the system is infinitely large and the magnetization can be non-zero even when h vanishes.

In a finite system the Hamiltonian is symmetric under the transformation $\sigma_i \to -\sigma_i$, and this causes the magnetization to be zero when $h = 0$ because

$$\langle M \rangle = \left. \frac{\partial \frac{\ln(Z)}{\beta N}}{\partial h} \right|_{h=0} = 0 \; \forall T$$

The solution to this problem is

[TODO: i fear this can be greatly expanded on]

1.3 Solutions

1.3.1 One dimensional case

The one dimensional Ising model is exactly solvable with and without an external magnetic field. The magnetization is given by:

[TODO: discuss solutions]

1.3.2 Two dimensional case

1.3.3 Three dimensional case

Chapter 2

Methods

2.1 Data representation

The code used in this project was written in python, using the Numba just-in-time compiler to achieve optimal execution speed (along best practices and algorithm optimizations in writing the code). From a development standpoint, every single Ising model configuration is represented in python as a d dimensional numpy array of shape N repeated d times.

```
1
2 def new_random_ising(N: tuple[int, ...]):
3     return np.ones(N, dtype = np.int8)
4
5 m = new_random_ising(tuple([N] * dim))
6
```

Note that the data type that the numpy array contains is np.int8. In order to minimize storage space, the idea of using boolean variables to represent 1 and -1 was considered, but was ultimately ditched in favour of storing integers. The latter grants faster executions when calculating physical observables (e.g.: energy, magnetization) at the loss of some disk space.

Every one of these numpy array is generated and stored in another numpy array of arbitrary length that represents the Markov Chain. This is done for a number of temperatures around the expected critical temperature, and is repeated for every lattice size under scrutiny and for every dimension. Ultimately, this data is stored in a hdf5 file for efficient i/o operations and in order to contextually add metadata and processed data (e.g.: autocorrelation time, thermalisation time, etc.) to the raw chain.

For every combination of dimension and lattice size, the generated row data is stored in a numpy array of shape $(N_{temps}, N_{steps}, [N_{dim}] \times d)$.

[TODO: maybe discuss floating point precision]

[TODO: definitely discuss the chunking of data to be loaded in memory during computation and the use of lzf compression]

2.2 Monte Carlo Markov Chains

2.2.1 Generation

To study a system at a given temperature it is necessary to be able to generate a random configuration of said system. These random configurations are distributed as a function of their energy: as discussed the probability of a system of being in a state with energy E in the canonical ensemble is given by:

$$P(E) = \frac{\Omega(E)e^{-\beta E}}{Z} \tag{2.1}$$

Now, it would be ideal to write this probability analytically: to do that we would need to know how to write both $\Omega(E)$ and Z, but but such an operation is, unfortunately, simply not possible. The lone computing of Z would require summing over all possible configurations of the system, which in turns would mean to compute the energy of every single configuration. The number of possible configurations of a (Nd) spins lattice is $2^{(Nd)}$ and computing the energy of a single configuration is an $O(N)$ operation, which means that the overall complexity of computing Z is $\mathcal{O}(N2^{(Nd)})$, which is clearly not viable for any reasonable value of N.

The solution to this problem is a Markov Chain Monte Carlo method, that is a method for generating a sequence of independent samples from a desired distribution without knowing the normalization constant. One algorithm that produces such a chain is the Metropolis-Hastings algorithm. This algorithm can draw samples from any probability distribution with probability density $P(x)$, provided that we know another function proportional to the density P that we're able to compute analitically (in this case $e^{-\beta E}$) so that we can write $P(E) \propto e^{-\beta E}$. The target distribution P is also called posterior.

The algorithm's idea is to start from a random configuration of the system and then to move to another configuration of the system with a set probability which is determined by the energy distribution.

The formal steps of the Metropolis algorithm in this use case are:

1. Start from a random configuration of the system x.
2. Randomly select a spin (or a number of spins) and flip it. We call this new configuration x'.
3. Compute the energy difference of the two systems ΔE.

4. Set an acceptance probability parameter α based on the energy change and the temperature: $\alpha = \frac{T(x' \to x)P(x')}{T(x \to x')P(x)}$ where P is the Boltzmann distribution. This is the core of the algorithm and it's what allows the Markov Chain to converge to the desired distribution.

5. Accept the new configuration with probability $\min(1, \alpha)$, otherwise reject it and keep the current configuration. That is, if the new configuration is more probable than the current one, accept it, otherwise accept it with probability α (that is, a probability proportional to the ratio of the two probabilities).

6. Add the current configuration to the Markov Chain.

7. Repeat steps 2-6 for a number of iterations. The elements of the chain after enough iterations are distributed as $e^{-\beta E}$

The "trick" that makes this algorithm work is that computing α both Z and the Ω terms cancel out.

[TODO: maybe discuss (formally or not) why the sample set given by the MC converges]

Note that randomly selecting a spin and flipping it (step 2) is effectively moving to another random configuration of the system close to the current one. This movement is intepreted as a random sample from a distribution centered around the current configuration. Picking a random spin to flip is an arbitrary choice, and other methods could be used to select a neighboring configuration, but has the advantage that the probability of moving from x to x' ($T(x \to x')$) is the same as moving from x' to x ($T(x' \to x)$), which is the Hastings condition of the algorithm. This condition is not striclty necessary, but it simplifies the computation of $\alpha = \frac{e^{-\beta E'}}{e^{-\beta E}} = e^{-\beta \Delta E}$ to be only dependent on the energy change and the temperature.

```
1
2  def metropolis_hastings(model: np.ndarray, T: float, steps: int):
3          '''
4          Generate a Markov Chain of configurations of the Ising model
           ↪  at a given temperature T.
5          '''
6
7      curr_model = np.copy(model)
8      models_samples = np.empty(
9                  (steps,) + curr_model.shape,
10                 dtype = curr_model.dtype
11         )
12
13         N = curr_model.shape[0]
14
15     for i in range(steps):
16         idx0 = np.random.randint(0, N)
17         spin = curr_model[idx0]
18         deltaE = compute_energy_change(curr_model, idx0)
```

```
19
20          accept_probability = np.minimum(1.0,
            ↪  target_distribution(deltaE, T))
21          if np.random.random() <= accept_probability:
22              curr_model[idx0] = -curr_model[idx0]
23
24          models_samples[i] = curr_model
25
26      return models_samples
27
```

Also note that since α depends only on the relative energy change the computation can be greatly optimized by computing only the energy change given by the single spin flip, instead of computing the energy of two entire configurations.

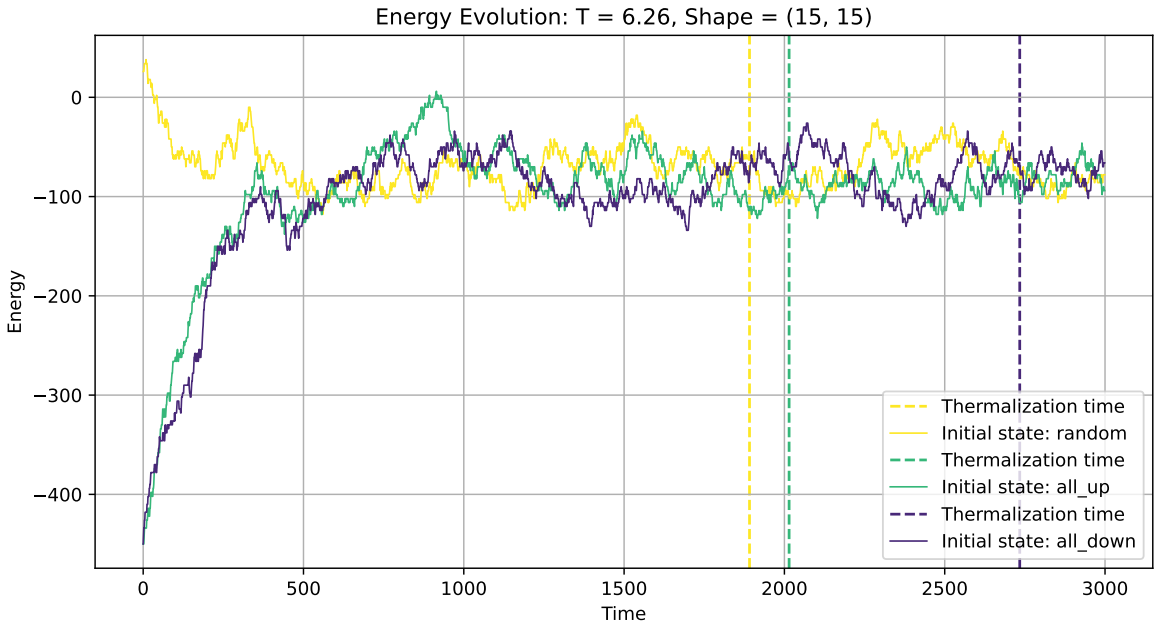


Figure 2.1: Markov Chain and relative distribution of the energy of the system.

In figure 2.1 some of the first steps of the chain can be seen as starting from different configurations (every spin set at random, all spins up, all spins down). Note that all of the chains start far from what should be the center of the distribution. This assures that before a single sample can be considered randomly sampled from the posterior some "adjusting" time needs to pass. More generally, such a chain suffers from some kind of "recency bias", meaning that, even though the exact previous step at any point of the chain is not ever identifiable, the approximate location (in energy) in of the configuration needs some time to be allowed to properly change. This is shown in the chains by the macro clusters of samples that remains in a narrow vertical space.

Figure 2.2 shows the very same chains after a much larger number of steps. It's clear that the chains have converged to a stationary distribution, and that the initial configuration is no longer relevant.

The energy distribution of the elements in the chain can now be appreciated to be converging to a normal distribution. This is not in contradiction with the fact that the distribution is supposed to be a Boltzmann distribution, but is in fact a consequence of the Central Limit Theorem (CLT), which for temperatures far from the critical temperature is applicable. On the contrary, close to T_C CLT is not applicable and the distribution diverges. [TODO: this can be expanded on]

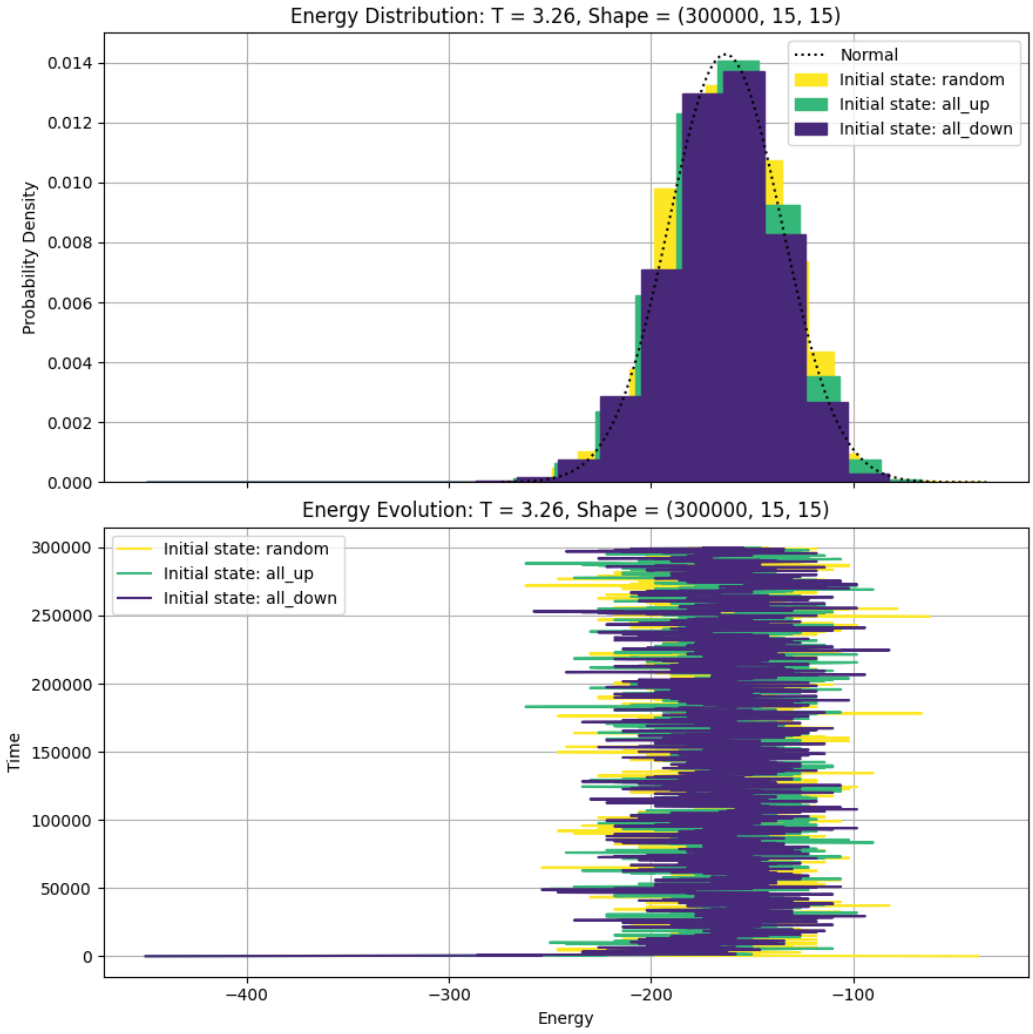


Figure 2.2: Markov Chain and relative distribution of the energy of the system.

2.2.2 Analysis

As stated, a Markov Chain suffers of a recency bias, and the set of configurations of the chain is not very useful as a real sample set since the elements are not independent: on the contrary, they are very closely correlated and the presence of correlations between samples would lead to biased results during analysis.

The extraction of decorrelated data is a problem composed of two parts: the thermalisation and the autocorrelation time. The first one is the number of steps that the Markov Chain needs to reach a stationary distribution, while the second one is the number of steps that need to exist between successive samples before they can be considered independent.

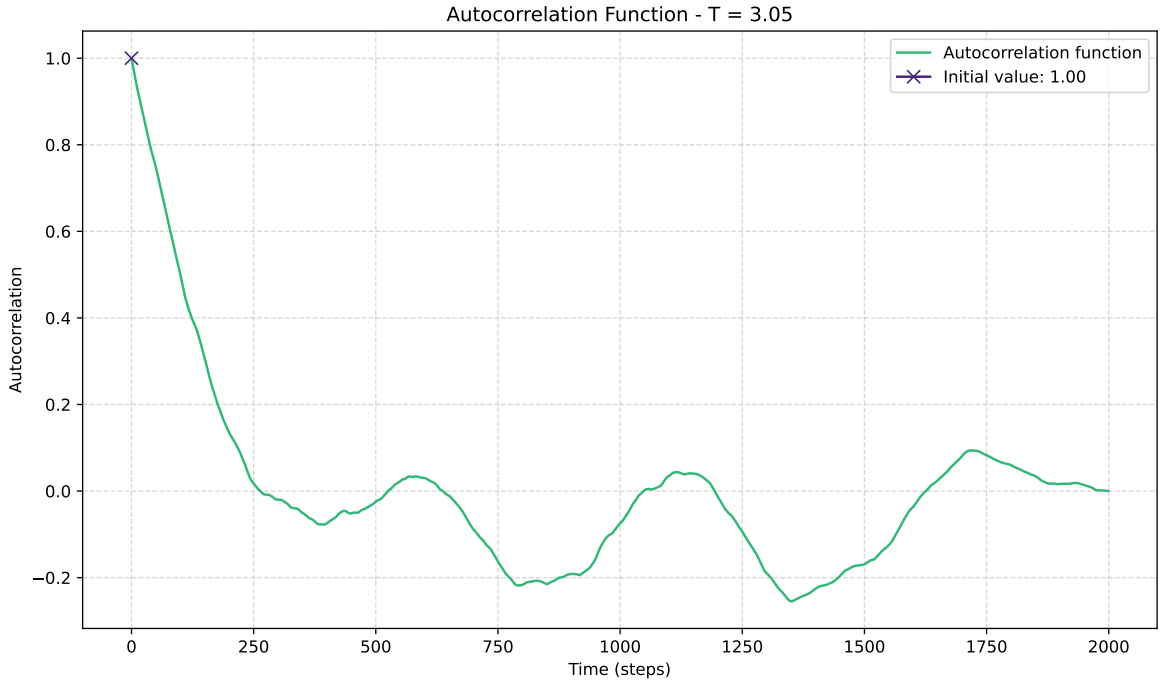


Figure 2.3: Example of autocorrelation function (of the energy) for a Markov Chain.

The plot in figure 2.3 shows the autocorrelation function of one Markov Chain. The autocorrelation function can be defined as:

$$\rho(t) = \frac{\mathbb{E}[(X_{t+\tau} - \mu)\overline{(X_t - \mu)}]}{\sigma^2} \tag{2.2}$$

where $\mathbb{E}$ denotes the expected value and X are the observable values measured on the configurations.

The following is the code used to extimate ρ from the Markov Chain:

```
1
2  def autocorrelation(t, observables: np.ndarray):
3          '''
4          Computes the autocorrelation function's value given:
5          - A time (step) t
6          - An array of observable values along the Markov chain
7          '''
8
9          N = len(observables)
```

```
10          if t >= N or t < 0:
11                  return 0.0
12
13          var = np.var(observables)
14
15          if var == 0:
16                  return 0.0
17
18          centered = observables - np.mean(observables)
19          autocov = np.sum(centered[:N - t] * centered[t:]) / N
20
21          return autocov / var
22
```

---

and it's computed as such for performance's sake. [TODO: maybe consider proving the formula]

The function ρ quantifies the similarity between observations of a random variable (energy, in this case) at different points in time, which can be used to develop methods to extract uncorrelated samples. Specifically, we see that ρ needs a transient time to reach a state of close-to-zero values and after this time it remains close to zero.

Two parameters can be defined from the autocorrelation function:

- The exponential autocorrelation time τ_{exp}, which is the time it takes for the autocorrelation function to decay into the equilibrium region.
- The integrated autocorrelation time τ_{int}, which is the sum of the autocorrelation function over all time lags, which is a measure of the overall correlation in the data. It's defined as
$$\tau_{\text{int}} = \frac{1}{2} + \sum_{t=1}^{\infty} \rho(t)$$
.

To sample uncorrelated data from the Markov Chain two numbers are needed:

- The number of steps to discard in front of the chain. This represent the thermalisation time, that is, the number of steps that the chain needs to reach a configuration that is representative of the stationary distribution at a certain temperature, in other words, the number of steps that the chain needs to "forget" its initial configuration. Conceptually, this is related to τ_{exp}.
- The number of steps to skip between samples (to avoid correlation between samples). This number is related to τ_{int}, and is usually set to a multiple of it (e.g.: $2\tau_{\text{int}}$).

A number of methods exist to estimate these two numbers, in this project only two are explored.

[TODO: I have the feeling that this portion can be rewritten better]

Note that theoretically speaking τ_{exp} can be neglected in long enough chains where we sample from a high enough multiple of τ_{int} (e.g. $1000\tau_{\text{int}}$), because the exponential decay will have already occurred. However, in practice it is always better to discard a number of steps in front of the chain to avoid the initial transient behavior. Discarding too many steps can be a problem too though, because it can lead to a loss of information and a decrease in the effective sample size (end relative errors). Practically, the number of steps to discard in this experiments will be set to $20\tau_{\text{int}}$, which is to be considered an extimation of τ_{exp}, and τ_{exp} itself will not be computed.

Sokal's method

A. Sokal describes a method to estime the integrated autocorrelation time τ_{int} in his lecture notes "Monte Carlo Methods in Statistical Mechanics: Foundations and New Algorithms" (1996). The following section discusses the method and its implementation in this project.

The natural estimator of τ_{int} would seem to be

$$\widehat{\tau}_{\text{int}} \equiv \frac{1}{2}\sum_{t=0}^{n}\widehat{\rho}(t) \tag{2.3}$$

but this is not really the case, because this estimator has a variance that does not go to zero as the sample size n goes to infinity, so it is clearly a somewhat bad estimator of τ_{int}. Qualitatively speaking, this happens because the sample autocorrelations $\widehat{\rho}(t)$ for big t contain much noise but little signal, and there are so many of them (order n) that the noise adds up to a total variance of order 1. The solution is to cut off the sum using a window $\lambda(t)$ which is ≈ 1 for small t but ≈ 0 for large t:

$$\widehat{\tau}_{\text{int}} \equiv \frac{1}{2}\sum_{t=0}^{n}\lambda(t)\widehat{\rho}(t). \tag{2.4}$$

This retains most of the signal but discards most of the noise. A good choice is the rectangular window

$$\lambda(t) = \begin{cases} 1 & \text{if } t \leq M \\ 0 & \text{if } t > M \end{cases} \tag{2.5}$$

where M is an arbitrarily chosen cutoff. The choice of M is a tradeoff between bias and variance: if M is too small, we throw away too much signal and the estimator is biased; if M is too large, we throw away too little noise and the estimator has a large variance. A good choice is to take M to be the first t for which $\widehat{\rho}(t)$ is negative.

This method is implemented in the following code:

```
1
2 def tau_int_sokal(observables, c = 15.0):
```

```
    '''
    Computes the integrated autocorrelation time using a self-consistent windowing method.
    c: The windowing parameter. The bigger c, the longer the window.
        longer window: tau has less bias
        shorter window: tau has less variance
    NOTE: higher c values yield more accurate results but require more computation time.
    '''

    N = len(observables)
    var = np.var(observables)

    if var == 0.0:
        return 0.0

    centered = observables - np.mean(observables)
    tau = 0.5

    for t in range(1, N):
        autocov_t = np.sum(centered[:N-t] * centered[t:]) / N
        tau += autocov_t / var

        if t >= c * tau:
            return tau

    return tau

```

Where the precise size of the window $M = c\tau$ is determined dinamically at runtime. [TODO: discuss why it's called a self-consistent method. Maybe also plot the autocorrelation function with the selected window.]

Data blocking method

While Sokal's method is based on summing over the autocorrelation function in a particular way, the data blocking method operates in a fundamentally different way. This method is a mathematical trick: let's consider a Markov Chain of length N of configurations x_i with variance σ^2 and an observable O mesured on each configuration. Now suppose to simplify this Chain by averaging the values of the observable O of every consecutive k elements. This will produce a new chain of length $N_b = N/k$ of block averages $\bar{O}_i$ that are less correlated than the original chain. One can then take advantage of the law:

$$V_k = \frac{\sigma^2}{k}(1 + 2\tau_{\text{int}}) \tag{2.6}$$

to build an estimator of τ_{int}:

$$\tau_{\text{int}}(k) = \frac{1}{2}(k\frac{V_k}{\sigma^2} - 1) \tag{2.7}$$

Note that 2.6 is intended to be a correction to the variance of the mean V_k for a chain of truly uncorrelated data:

$$V_k = \frac{1}{N_b - 1}\sum_{i=1}^{N_b}(\bar{O}_i - \bar{O})^2 = \frac{\sigma^2}{k} \tag{2.8}$$

and the correction factor can be written as a function of τ_{int} only in the limit $N \gg \tau_{\text{int}}$. To verify that these laws are consistent with each other, one can see that for truly uncorrelated data $\tau_{\text{int}} \to 0$, and so equation 2.6 reduces to equation 2.8 for $N = k$ (or $N_b = 1$ or $N \gg \tau_{\text{int}}$).

Now, the idea is that since $\tau_{\text{int}} = \tau_{\text{int}}(k)$ it can be studied as a function of k. This can be done by iterating the simplification process described above on the chain in order to create a coarser chain at every iteration, where $\bar{O_i}(k)$ are less correlated at every iteration

The blocks will contain k elements, and their size will double at every iteration. This means that for every iteration the number of blocks N_b will be halved and k will be doubled. The first iteration is $k = 1$ and $N_b = N$.

For every k the procedure is as follows:

1. Divide the chain into N_b blocks of size k. The number of blocks is then $N_b = N/k$ and compute the average of the observable O in each block. This is equivalent to averaging the observables of every two consecutive blocks of the previous iteration.
2. Compute the variance of the new set of block averages.
3. Compute $\tau_{\text{int}}(k)$ as $\tau_{\text{int}}(k) = \frac{1}{2}(k\frac{V_k}{\sigma^2} - 1)$.

The estimate $\tau_{\text{int}}(k)$ will converge to the true value of τ_{int} as k increases.

After a certain value of k the limit $k \gg \tau_{\text{int}}$ will be reached (in a numerical sense) and the estimate for τ_{int} will converge to a plateau (because V_k will be stable). The value of the plateau is then taken as the final estimate of τ_{int}.

It's important to note that this method can be used on a chain that has reached the stationary distribution, from the beginning. So, in the actual implementation, the thermalisation time is computed as $20\tau_{\text{int}}^{\text{Sokal}}$ estimated with Sokal's method, and then the data blocking method is applied to the chain after discarding the first $20\tau_{\text{int}}^{\text{Sokal}}$ steps.

The implementation is shown in the following code:

```python

def tau_int_blocking(observables, burn_in = None):
    '''
    Computes the integrated autocorrelation time using the data blocking method.
    observables: ndarray of observable values along the Markov chain.
    burn_in: int, optional
        The number of initial samples to discard as burn-in.
        if None, burn_in is estimated as 20 tau_int computed with the Sokal method.
    '''

    N = len(observables)

    if burn_in is None:
        tau0 = tau_int_sokal(observables, c = 5.0)

    observables = observables[burn_in:]
    N = len(observables)

    var = np.var(observables)

    current_blocks = observables.copy()
    tau_estimates = []
    block_size = 1 # k
    Nb_array = [] # Number of blocks for each block size
    Nb = N # Number of blocks for the current block size (Nb_array[0])

    while Nb >= 2:
        # Estimation
        mean_blocks = np.mean(current_blocks)
        var_blocks = np.sum(np.power(current_blocks - mean_blocks, 2)) / (Nb - 1)
        tau_estimate = .5 * (block_size * (var_blocks / var) - 1)
        tau_estimates.append(tau_estimate)
        Nb_array.append(Nb)

        # Blocking
        if Nb % 2 != 0:
            current_blocks = current_blocks[:-1]  # Discard the last block if odd number of blocks
            Nb -= 1
        Nb //= 2
        current_blocks = 0.5 * (current_blocks[::2] + current_blocks[1::2])
```

```
        block_size *= 2

    for i in range(len(tau_estimates) - 1):
        if Nb_array[i] < 30:
            return float(tau_estimates[i])

        diff = abs(tau_estimates[i + 1] - tau_estimates[i])
        err = (tau_estimates[i] + 0.5) * np.sqrt(2.0 / (Nb_array[i] -
        ↪  1))

        if diff < err:
            return float(tau_estimates[i])

    return float(tau_estimates[-1])

```

Chapter 3

Results

[TODO: Discuss in general the graphs of M in the contest of a phase transition?]

3.1 1D Ising model

The one dimensional Ising model is known not to present a phase transition at any finite temperature ($T_C > 0$).

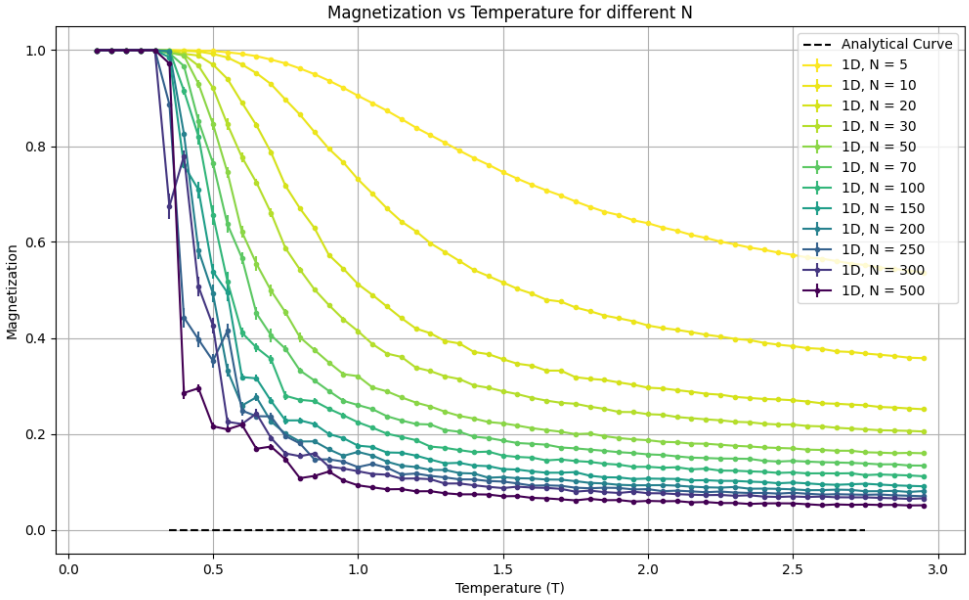


Figure 3.1: Magnetization for the 1D Ising model.

What can be seen in figure 3.1 is that the magnetization $M(T)$ approaches a constantly null function as N increases. This is consistent with the theoretical prediction. The value close to 1 are simply an effect of finite-size corrections: the critical temperature would be exactly 0 with infinite size. The reason why increasing the size of the system

leads to the movement of the "step" towards 0 is that the energy cost of breaking a streak of aligned spins is independent of the size of the system, which means that for bigger N the probability of having a single big streak of aligned spins is lower, and for every streak of aligned-up spins there is a high probability a streak of a same length of aligned-down spins to exist, which leads to a null magnetization. Since the flipping of a spin is an effect of thermal fluctuations the magnetization is closer to zero for higher temperatures (this can be seen in the curves of the smaller lattices: they don't drop instantly but smoothly curve towards zero), but this effect is negligible for bigger lattices, where the magnetization drops to zero faster.

3.2 2D Ising model

The two dimensional Ising model presents a nicer result, as it is known to present a phase transition at a finite temperature. This is directly translated in the approach to a step function of the magnetization $M(T)$ as N increases, which can be seen in figure 3.2.

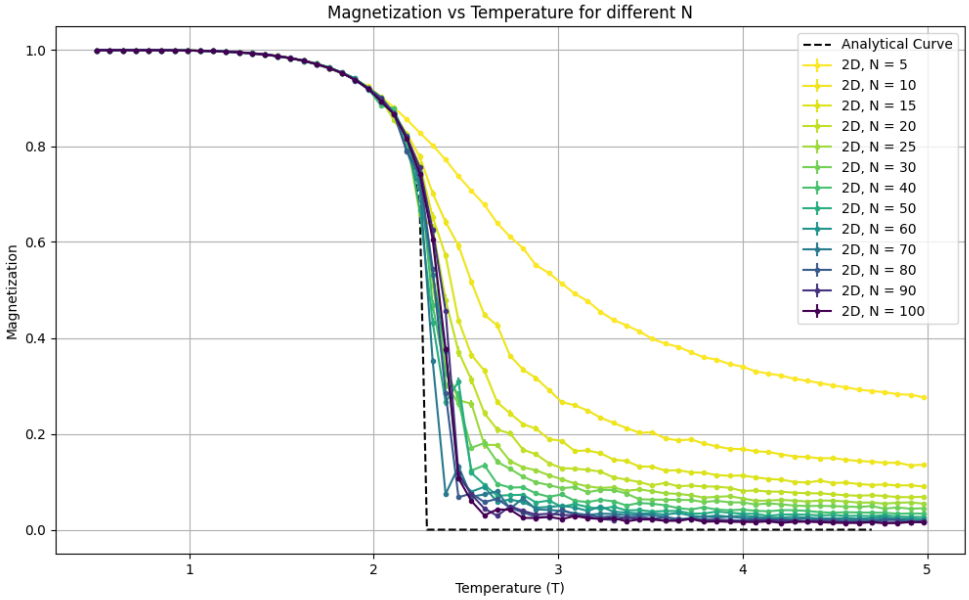


Figure 3.2: Convergence of magnetization for the 2D Ising model.

The accepted value for the critical temperature of the 2D Ising model is $T_C \approx 2.269$. [TODO: add extracted value of T_C]

The difference between the 1D and 2D Ising model is that in the 2D case a single spin flip is enough to break a streak of aligned spins, that is, the a single spin flip che change significantly the contribution to the magnetiziation of a section of the lattice. The equivalent of a streak of aligned spins in 2D is a cluster of aligned spins, and the equivalent of a single spin flip to break the streak is the flipping of the spins on the border of the cluster. The energy needed to flip a single spin is still independent of the

size of the system, but the size of the perimeter of a cluster is not, and this introduces a size dependence in the probability of having a cluster of aligned spins, which is what leads to a threshold behavior.

3.3 3D Ising model

The same argument for the 2D Ising model can be applied to the 3D case, and the results are similar. The accepted value for the critical temperature of the 3D Ising model is $T_C \approx 4.5115$. [TODO: add extracted value of T_C]

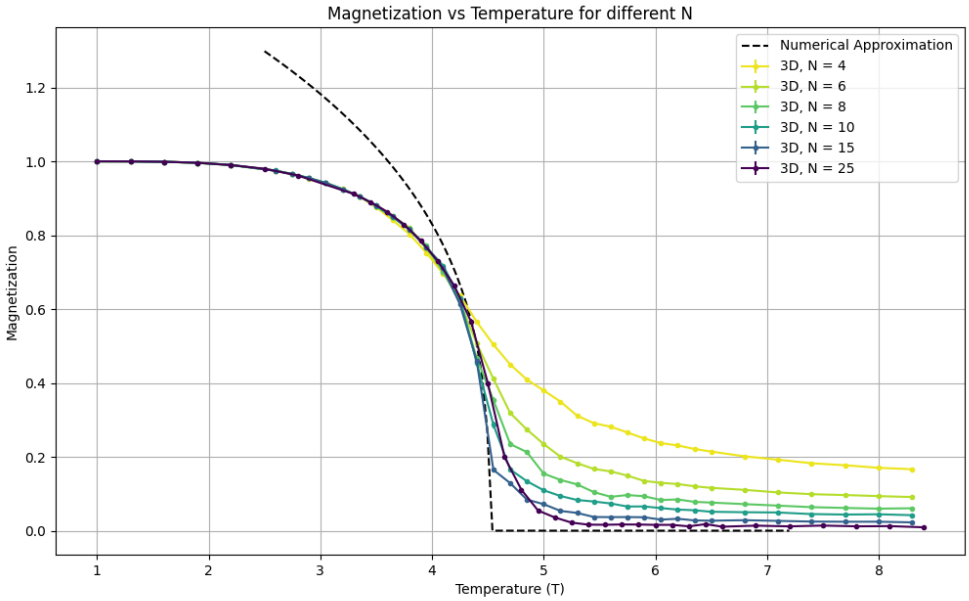


Figure 3.3: Convergence of magnetization for the 3D Ising model.

The theoretical difference is that the 3D model is not analytically solved (in the plot the numerical results are confronted with a critical scaling behavior [TODO: fonte maybe A L Talapov and H W J Blöte 1996 J. Phys. A: Math. Gen. 29 5727]), but the existence of a phase transition is well-established, and the numerical results are consistent with the expected behavior.

Appendix A

Appendix

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

Bibliography

[1] Author, Title, Journal/Book, Year.